CS 374: Section A - Homework 5, Problem 2

Anirudh Konidala

October 6, 2026

The ith element has value $S[i]$ and weight $W[i]$, values and weights are positive and distinct, and $w(R)$ is the total weight of a set R. The weighted median of S is the element x with

$$w(S_{<x}) \leq \frac{w(S)}{2} \quad \text{and} \quad w(S_{>x}) < \frac{w(S)}{2}.$$

The algorithm only compares values and adds or subtracts weights; it uses no hashing and no radix sort. We use the linear-time selection algorithm from class (MomSelect, median of medians) as a black box, and the Partition subroutine from quicksort, modified to move each weight $W[i]$ along with its value $S[i]$.

A more general problem

Since $w(S_{>x}) = w(S) - w(S_{<x}) - w(x)$, the second condition is the same as $\frac{w(S)}{2} < w(S_{<x}) + w(x)$. So the weighted median is the unique x with

$$w(S_{<x}) \leq \frac{w(S)}{2} < w(S_{<x}) + w(x).$$

In words: lay the elements end to end in order of value on the number line, each covering an interval as long as its weight; the weighted median is the element whose interval contains the point $\frac{w(S)}{2}$. We solve the same problem for an arbitrary point t.

Weighted selection: WSelect$(S[1..n], W[1..n], t)$, given a non-empty weighted set and a number t with $0 \leq t < w(S)$, returns the value of the unique element x with $w(S_{<x}) \leq t < w(S_{<x}) + w(x)$.

This x exists and is unique: the intervals $[w(S_{<x}), w(S_{<x}) + w(x))$, taken in order of value, are disjoint, have positive length, and together cover $[0, w(S))$ exactly.

```
WSelect(S[1..n], W[1..n], t):
    if n = 1
        return S[1]
    v ← MomSelect(S[1..n], ⌈n/2⌉)          ⟨⟨median value⟩⟩
    find the index p with S[p] = v
    r ← Partition(S[1..n], W[1..n], p)      ⟨⟨pivot v now at S[r]⟩⟩
    w_L ← W[1] + W[2] + ··· + W[r − 1]      ⟨⟨weight of smaller values⟩⟩
    if t < w_L
        return WSelect(S[1..r − 1], W[1..r − 1], t)
    else if t < w_L + W[r]
        return S[r]
    else
        return WSelect(S[r + 1..n], W[r + 1..n], t − w_L − W[r])
```

Weighted median: WeightedMedian$(S[1..n], W[1..n])$ returns the value of the weighted median of the given weighted set.

```
WeightedMedian(S[1..n], W[1..n]):
    total ← 0
    for i ← 1 to n
        total ← total + W[i]
    return WSelect(S[1..n], W[1..n], total /2)
```

The call satisfies the precondition of WSelect, because $0 \le \frac{w(S)}{2} < w(S)$ when all weights are positive. By the reformulation above, the element WSelect returns for $t = \frac{w(S)}{2}$ is exactly the weighted median.

Why WSelect is correct

We argue by induction on n. When $n = 1$ the only element x has $w(S_{<x}) = 0 \le t < w(x) = w(S)$, so returning it is correct.

For $n \ge 2$, after partitioning let $L = S[1..r-1]$ hold the values smaller than the pivot $S[r]$ and $G = S[r+1..n]$ hold the larger ones, so that $w_L = w(L)$ and $w(S) = w_L + W[r] + w(G)$.

- **If $t < w_L$.** Then L is non-empty and $0 \le t < w(L)$, so the recursive call satisfies the precondition. For every $x \in L$ the elements of S smaller than x all lie in L, so $S_{<x} = L_{<x}$, and the element that L assigns to t is the element that S assigns to t.
- **If $w_L \le t < w_L + W[r]$.** The pivot $x = S[r]$ has $S_{<x} = L$, so $w(S_{<x}) \le t < w(S_{<x}) + w(x)$ and the pivot is the answer.
- **If $t \ge w_L + W[r]$.** Let $t' = t - w_L - W[r]$. Then $t' \ge 0$, and $t' < w(S) - w_L - W[r] = w(G)$, so G is non-empty and the recursive call satisfies the precondition. For every $x \in G$ we have $S_{<x} = L \cup \{S[r]\} \cup G_{<x}$, so $w(S_{<x}) = w_L + W[r] + w(G_{<x})$. Subtracting $w_L + W[r]$ from every

part of the condition $w(S_{<x}) \le t < w(S_{<x}) + w(x)$ turns it into $w(G_{<x}) \le t' < w(G_{<x}) + w(x)$, so the element that G assigns to t' is the element that S assigns to t.

In every case WSelect returns the correct element, assuming by induction that the recursive call on the smaller set L or G is correct.

Running time

MomSelect runs in $O(n)$ time, and finding p, Partition, and the sum w_L each take one pass over the array, so one call of WSelect does $O(n)$ work besides its recursive call. Because the pivot is the median, L and G each hold at most $\frac{n}{2}$ elements, and WSelect makes at most one recursive call. So, ignoring floors and ceilings,

$$T(n) \le T\Big(\frac{n}{2}\Big) + O(n),$$

which solves to $T(n) = O(n)$, since the work per level shrinks geometrically: $O(n + \frac{n}{2} + \frac{n}{4} + \cdots) = O(n)$. WeightedMedian adds one $O(n)$ pass to compute the total weight, so the whole algorithm runs in

$$\boxed{O(n) \text{ time.}}$$

This beats the $O(n \log n)$ algorithm that sorts by value and scans the prefix sums of the weights.

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) to draft this algorithm and explain its structure to me. I traced it by hand on several small weighted sets against the answer from sorting, and typeset the submission myself.